2026 Spring

Course ID: 1142-1765

AdvancedProgramming

03-Python programming for “Plus One”

Advanced Programming

Python · C++ · Rust

Data Structures & Algorithms

Python | C++ | Rust

Dynamic Programming · Divide & Conquer · Greedy Algorithms · Arrays · Lists · Maps & Sets

Part-1

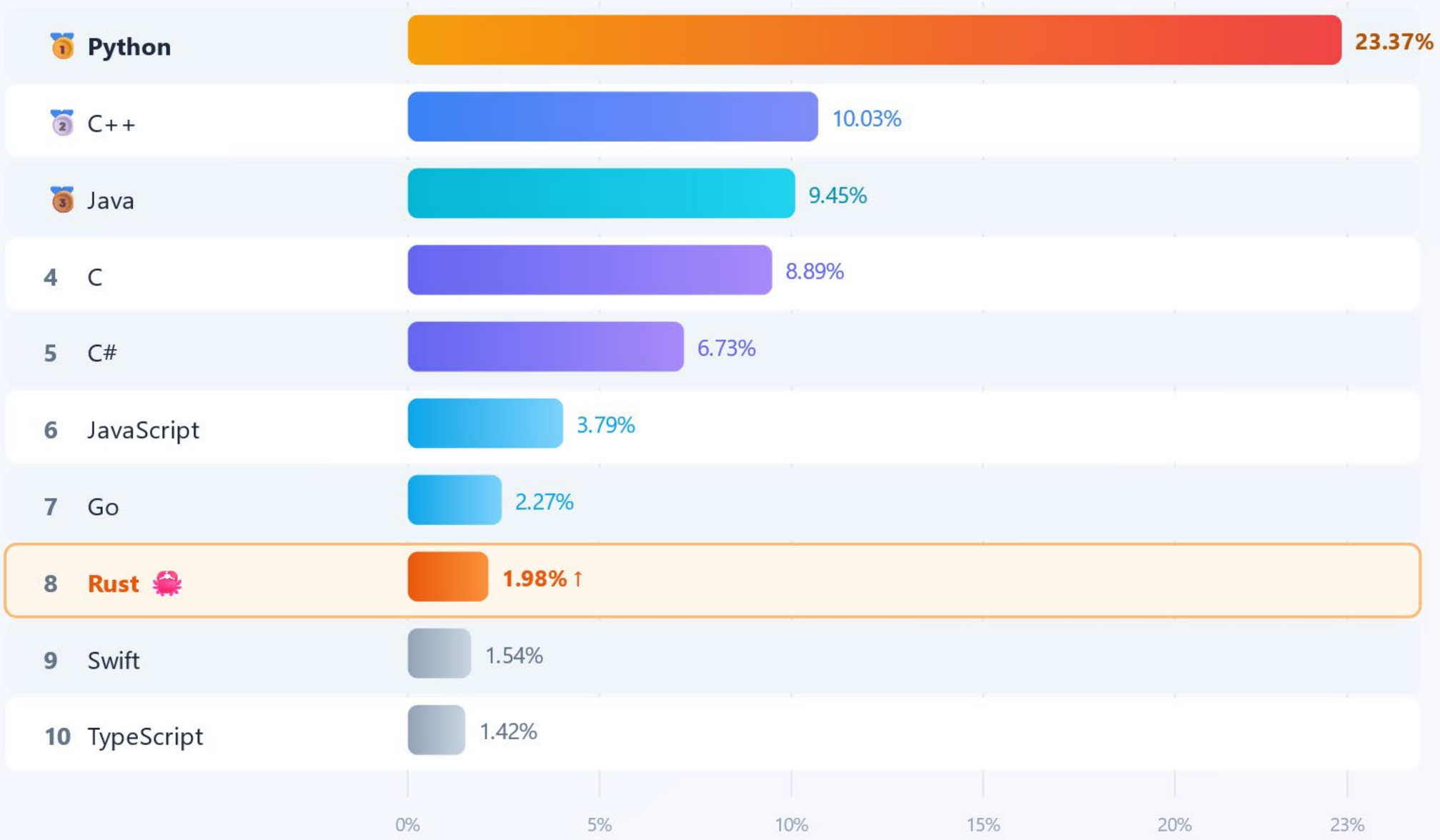
Python Data Structures

LeetCode Problem
"Plus One"

Course Activities
Github repository
Discussion Forum

🏆 Top 10 Programming Languages

Source: TIOBE Index · November 2025



🦀 Rust reached its all-time high of #8 in Nov 2025 — memory safety & performance driving adoption

Python
Data Structures

dict · list · set · tuple

Comprehensive Guide to Python's Built-in Data Structures

Overview: Python's Core Data Structures

[ ]

List

Ordered, mutable sequence of items

( )

Tuple

Ordered, immutable sequence of items

{ : }

Dictionary

Key-value pairs, mutable mapping

{ }

Set

Unordered unique elements collection

Property	List	Tuple	Dict	Set
Ordered	✓	✓	✓ (3.7+)	✗
Mutable	✓	✗	✓	✓
Duplicates	✓	✓	Keys: ✗	✗
Indexed	✓	✓	By key	✗

List — Ordered & Mutable Sequence

What is a List?

1. Stores an ordered collection of items
2. Elements can be of any data type
3. Mutable: add, remove, or change elements
4. Supports duplicate values
5. Uses zero-based integer indexing
6. Supports negative indexing (-1 = last)
7. Dynamic: grows or shrinks automatically

```
# Creating lists
fruits = ["apple", "banana", "cherry"]
mixed  = [1, "hello", 3.14, True]
nested = [[1, 2], [3, 4], [5, 6]]
empty  = []

# Indexing
print(fruits[0])     # apple
print(fruits[-1])    # cherry

# Slicing  [start:stop:step]
print(fruits[0:2])   # ['apple', 'banana']
print(fruits[::-1])  # reverse the list
```

💡 Memory Tip

Lists are stored as dynamic arrays in CPython. Appending is O(1) amortized, but insertion at position 0 is O(n).

List — Common Methods & Operations

Method	Description	Complexity
`append(x)`	Add x to end	O(1)
`insert(i, x)`	Insert x at index i	O(n)
`extend(iter)`	Add all items from iter	O(k)
`remove(x)`	Remove first x	O(n)
`pop([i])`	Remove & return item	O(1)
`sort()`	Sort in place	O(n log n)
`reverse()`	Reverse in place	O(n)
`index(x)`	Find first index of x	O(n)
`count(x)`	Count occurrences of x	O(n)
`copy()`	Shallow copy	O(n)

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]

nums.append(7)        #
[3,1,4,1,5,9,2,6,7]
nums.insert(0, 0)     # [0,3,1,4,...]
nums.remove(1)        # removes first 1
last = nums.pop()     # removes 7
nums.sort()           # in-place sort
nums.sort(reverse=True) # descending

# Sorted (returns new list)
s = sorted(nums, key=lambda x: -x)

# List as stack
stack = []
stack.append("push")
top = stack.pop()

# List as queue (use deque instead!)
from collections import deque
q = deque([1, 2, 3])
q.appendleft(0)
q.popleft()
```

List — Comprehensions & Advanced Patterns

Syntax: `[expression for item in iterable if condition]`

Fast, Pythonic way to build lists in a single readable line

```
# Basic comprehension
squares = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# With condition (filter)
evens = [x for x in range(20) if x % 2 == 0]

# Transform strings
words  = ["hello", "world", "python"]
upper  = [w.upper() for w in words]

# Nested comprehension (flatten)
matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat   = [n for row in matrix for n in row]
```

```
# Conditional expression (ternary)
labels = ["even" if x%2==0 else "odd"
          for x in range(8)]

# Dict from two lists
keys   = ["a", "b", "c"]
vals   = [1, 2, 3]
d = {k: v for k, v in zip(keys, vals)}

# Set comprehension
unique_lens = {len(w) for w in words}

# Generator (memory efficient)
gen = (x**2 for x in range(1_000_000))
total = sum(gen)    # No list created!
```

Tuple — Ordered & Immutable Sequence

What is a Tuple?

1. Ordered sequence — like a list
2. Immutable: cannot modify after creation
3. Faster than lists for read operations
4. Can be used as dictionary keys
5. Ideal for fixed data (coordinates, RGB)
6. Supports packing & unpacking
7. Uses less memory than a list

```
# Creating tuples
point   = (3, 7)
rgb     = (255, 128, 0)
single  = (42,)       # Note: comma required!
empty   = ()
no_paren = 1, 2, 3    # Implicit packing

# Indexing & slicing (same as list)
print(point[0])       # 3
print(rgb[-1])        # 0

# Immutability
point[0] = 5   # ❌ TypeError!

# But can contain mutable objects
t = ([1, 2], [3, 4])
t[0].append(99)       # ✅ Works!
print(t)   # ([1, 2, 99], [3, 4])
```

List vs Tuple Speed:

tuple creation is ~2× faster; tuple iteration is ~10–20% faster than list

Tuple — Unpacking, Swapping & Named Tuples

```
# Basic unpacking
x, y = (3, 7)
a, b, c = (10, 20, 30)

# Swap variables (no temp needed!)
a, b = b, a

# Extended unpacking (Python 3)
first, *rest = (1, 2, 3, 4, 5)
# first=1, rest=[2, 3, 4, 5]

*head, last = (1, 2, 3, 4, 5)
# head=[1,2,3,4], last=5

# Ignore values with _
_, important, _ = ("skip", "keep", "skip")
```

```
from collections import namedtuple

# Define named tuple type
Point = namedtuple("Point", ["x", "y"])
Color = namedtuple("Color", ["r", "g", "b"])

p = Point(3, 7)
print(p.x, p.y)   # 3 7
print(p[0])       # 3 (index still works)

c = Color(255, 128, 0)
print(c.r)        # 255

# Use in function returns
def get_stats(data):
    Stats = namedtuple("Stats",
["min","max","avg"])
    return Stats(min(data), max(data),
```

Function Returns

Return multiple values cleanly

Dict Keys

Tuples are hashable; lists are not

Data Integrity

Protect data from accidental change

Pattern Matching

Use in match-case (Python 3.10+)

Dictionary — Key-Value Mapping

What is a Dictionary?

1. Stores key-value pairs (mappings)
2. Keys must be unique & hashable
3. Values can be any Python object
4. Ordered by insertion (Python 3.7+)
5. O(1) average lookup by key
6. Implemented as a hash table
7. Most used data structure in Python

```
# Creating dictionaries
person = {"name": "Alice", "age": 30}
empty  = {}
from_keys = dict.fromkeys(["a","b"], 0)
zipped = dict(zip(["x","y"], [1, 2]))

# Accessing values
print(person["name"])       # Alice
print(person.get("age"))    # 30
print(person.get("x", -1)) # -1 (default)

# Modifying
person["age"] = 31
person["city"] = "Taipei"  # add new key
del person["city"]          # delete key

# Check existence
if "name" in person:
    print("Found!")
```

⚡ Performance: dict lookup is O(1) average. Perfect for counters, caches, configs, and lookup tables.

Dictionary — Methods & Iteration

```python
d = {"a": 1, "b": 2, "c": 3}

# View objects (dynamic!)
d.keys()    # dict_keys(['a','b','c'])
d.values()  # dict_values([1, 2, 3])
d.items()   # dict_items([('a',1)...])

# Iterate
for key in d:
    print(key, d[key])

for key, val in d.items():
    print(f"{key} → {val}")

# Update / Merge
d.update({"d": 4, "e": 5})
merged = {**d, "f": 6}     # unpack merge
merged = d | {"f": 6}      # Python 3.9+

# Safe pop
val = d.pop("a")            # removes & returns
val = d.pop("z", "none")    # no KeyError
```

```python
# setdefault — get or create
counts = {}
for word in ["hi", "hi", "bye"]:
    counts.setdefault(word, 0)
    counts[word] += 1

# defaultdict (auto-create values)
from collections import defaultdict
graph = defaultdict(list)
graph["A"].append("B")      # No KeyError!

# Counter (specialized dict)
from collections import Counter
text = "abracadabra"
c = Counter(text)
# Counter({'a':5,'b':2,'r':2,'c':1,'d':1})
c.most_common(3)  # top 3 elements

# Nested dict
config = {
    "db": {"host": "localhost", "port": 5432},
    "app": {"debug": True, "port": 8000}
}
```

📝 **Dict Comprehension:** `{k: v*2 for k, v in d.items() if v > 1}` → filter & transform in one line

Set — Unique Unordered Collection

What is a Set?

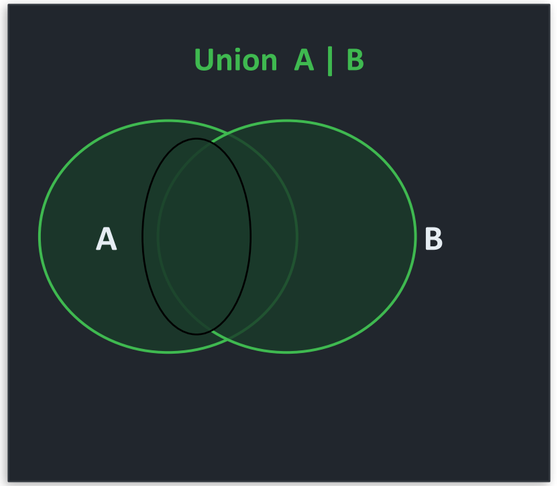
1. Unordered collection — no indexing
2. All elements are unique (no duplicates)
3. Elements must be hashable (immutable)
4. O(1) average membership testing
5. Supports mathematical set operations
6. Mutable: add/remove elements
7. frozenset is the immutable variant

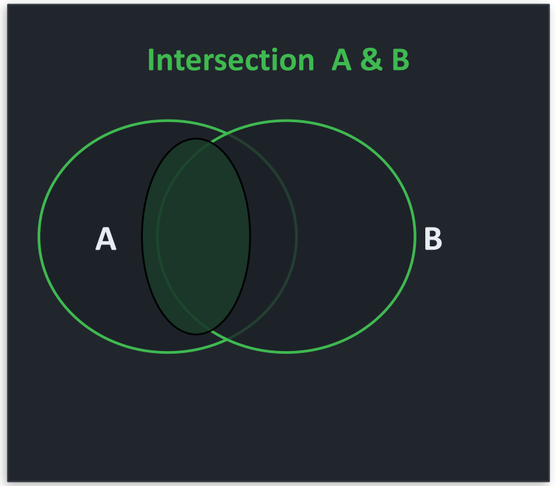
```
# Creating sets
fruits  = {"apple", "banana", "cherry"}
nums    = {1, 2, 3, 4, 5}
from_list = set([1, 2, 2, 3, 3, 3])  # {1,2,3}
empty   = set()  # ⚠️  NOT {}  (that's a dict!)

# Membership test (O(1) — very fast!)
print("apple" in fruits)    # True
print("grape" in fruits)    # False

# Adding & removing
fruits.add("grape")
fruits.discard("mango")     # no error if absent
fruits.remove("banana")     # KeyError if absent
popped = fruits.pop()       # remove random

# No indexing!
fruits[0]   # ❌ TypeError
```

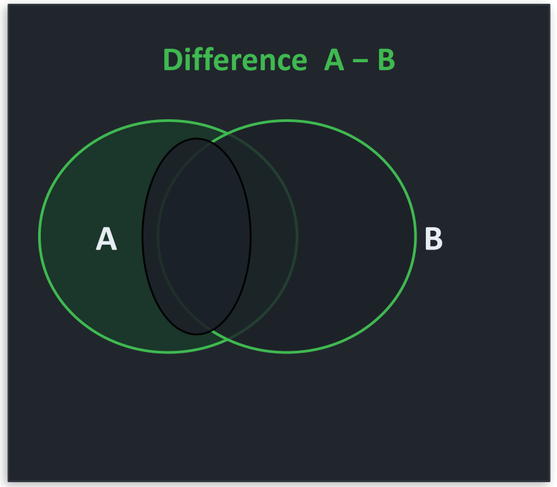
💡 Best use: fast duplicate removal → unique = set(my_list) or list(set(my_list))

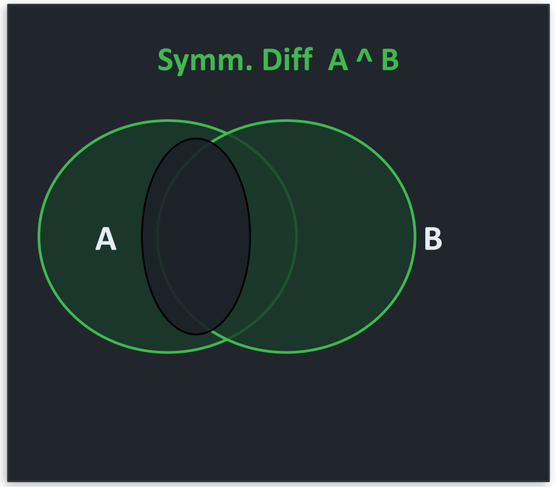
Set — Mathematical Operations

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# Union  (A | B)  — all elements
A | B           # {1,2,3,4,5,6,7,8}
A.union(B)      # same result

# Intersection  (A & B)  — common
A & B           # {4, 5}
A.intersection(B)

# Difference  (A - B)  — in A not B
A - B           # {1, 2, 3}
A.difference(B)

# Symmetric difference  (A ^ B)
A ^ B           # {1,2,3,6,7,8}
A.symmetric_difference(B)

# Subset / superset checks
{1,2} <= A      # True  (subset)
{1,2}.issubset(A)
A >= {1,2}      # True  (superset)
A.issuperset({1,2})
A.isdisjoint({9,10})  # True (no overlap)
```









Set — frozenset & Practical Patterns

```
# frozenset — immutable set
fs = frozenset([1, 2, 3, 4])
fs.add(5)      # ❌ AttributeError

# Use as dict key (hashable!)
graph = {
    frozenset({0,1}): "edge A-B",
    frozenset({1,2}): "edge B-C"
}

# Use as set element
s = {frozenset({1,2}), frozenset({3,4})}

# Practical: Remove duplicates (preserve order)
seen = set()
unique = [x for x in [1,2,2,3,1,4]
          if not (x in seen or seen.add(x))]
# [1, 2, 3, 4]
```

```
# Fast membership testing  vs  list
big_list = list(range(1_000_000))
big_set  = set(range(1_000_000))

# in list → O(n) scans up to 1M items
999999 in big_list   # slow

# in set  → O(1) hash lookup
999999 in big_set    # instant!

# Common patterns
# 1. Deduplication
tags = ["py", "ml", "py", "ai", "ml"]
unique_tags = list(set(tags))

# 2. Find common items
admins = {"alice", "bob"}
logged_in = {"bob", "carol", "alice"}
online_admins = admins & logged_in
```

Deduplication

Fastest way to remove duplicates from any iterable

Membership Test

O(1) check vs O(n) for lists — use sets for large lookups

Set Algebra

Union, intersection, difference for data reconciliation

Graph Theory

Model nodes, edges, visited states in BFS/DFS

Nesting & Combining Data Structures

```python
# List of dicts (very common!)
students = [
    {"name": "Alice", "grade": 92},
    {"name": "Bob",   "grade": 85},
    {"name": "Carol", "grade": 78}
]
top = [s for s in students if s["grade"] >= 90]

# Dict of lists
schedule = {
    "Mon": ["Math", "English"],
    "Tue": ["Physics", "Art"],
}
schedule["Mon"].append("PE")

# Dict of dicts
config = {
    "db":  {"host": "db.local", "port": 5432},
    "api": {"host": "api.local", "port": 8080},
}
```

```python
# Tuple as dict key (x, y) → value
grid = {}
grid[(0, 0)] = "start"
grid[(3, 4)] = "goal"

# Set of tuples
visited = set()
visited.add((0, 0))
visited.add((1, 2))
if (0, 0) in visited:
    print("Already visited!")

# JSON-like nested structure
data = {
    "users": [
        {"id": 1, "tags": {"admin","staff"}},
        {"id": 2, "tags": {"user"}},
    ],
    "total": 2
}
```

Design Principle: Choose the outermost structure by access pattern. Use dicts for named fields, lists for ordered sequences, sets for uniqueness checks, tuples for immutable records.

Comprehensions — All Four Variants

List Comprehension

```
# [expression for x in iter if cond]
squares = [x**2 for x in range(10)]
evens   = [x for x in range(20) if x%2==0]
# Returns: list
```

Dict Comprehension

```
# {key: value for x in iter if cond}
sq_map  = {x: x**2 for x in range(5)}
# {0:0, 1:1, 2:4, 3:9, 4:16}
# Returns: dict
```

Set Comprehension

```
# {expression for x in iter if cond}
unique  = {x%5 for x in range(20)}
# {0, 1, 2, 3, 4}
# Returns: set
```

Generator Expression

```
# (expression for x in iter if cond)
gen = (x**2 for x in range(1_000_000))
total = sum(gen)  # Memory efficient!
# Returns: generator (lazy)
```

⚡ Performance Ranking (fastest → slowest): Generator < Set comp ≈ Dict comp < List comp (generators never build the full collection)

Mutability, Hashability & Memory

```
# Mutability demo
lst   = [1, 2, 3]
lst.append(4)        # ✅ mutable

tup   = (1, 2, 3)
tup[0] = 9           # ❌ TypeError

dct   = {"a": 1}
dct["b"] = 2         # ✅ mutable

st    = {1, 2, 3}
st.add(4)            # ✅ mutable

# References vs copies
a = [1, 2, 3]
b = a               # same object!
b.append(4)
print(a)            # [1, 2, 3, 4] !!

c = a.copy()        # shallow copy
d = a[:]            # also shallow copy
import copy
e = copy.deepcopy(a)  # deep copy
```

```
# Hashability (required for dict keys / set
items)
hash("hello")      # ✅ str is hashable
hash((1, 2, 3))    # ✅ tuple is hashable
hash(42)           # ✅ int is hashable
hash([1, 2, 3])    # ❌ TypeError: unhashable

# Memory sizes (approximate)
import sys
lst = [1, 2, 3, 4, 5]
tup = (1, 2, 3, 4, 5)
dct = {1:"a", 2:"b", 3:"c", 4:"d", 5:"e"}
st  = {1, 2, 3, 4, 5}

sys.getsizeof(lst)   # ~104 bytes
sys.getsizeof(tup)   # ~80  bytes (smaller!)
sys.getsizeof(dct)   # ~360 bytes
sys.getsizeof(st)    # ~216 bytes
```

Type	Mutability	Hashable?	As Key?
list	✓ Mutable	✗ Unhashable	dict key ✗
tuple	✗ Immutable	✓ Hashable	dict key ✓
dict	✓ Mutable	✗ Unhashable	dict key ✗

Comprehensive Comparison — All Four Structures

Feature	list [ ]	tuple ( )	dict { : }	set { }
Syntax	[1,2,3]	(1,2,3)	{"a":1}	{1,2,3}
Ordered	✓ Yes	✓ Yes	✓ 3.7+	✗ No
Mutable	✓ Yes	✗ No	✓ Yes	✓ Yes
Duplicates	✓ Yes	✓ Yes	Keys: ✗	✗ No
Indexable	✓ int index	✓ int index	✓ by key	✗ No
Hashable	✗ No	✓ Yes	✗ No	✗ No
Lookup O(n)	`O(n)`	`O(n)`	`O(1) avg`	`O(1) avg`
Use case	Ordered data	Fixed records	Mappings	Unique items
Memory	Moderate	Low	High	Moderate
Empty create	`list() / []`	`tuple() / ()`	`dict() / {}`	`set()`

When to Use Which Data Structure?

Use LIST when…

- Order matters and may change
- You need indexed access by position
- Duplicates are expected/needed
- Building a stack or queue
- Sorting or filtering a collection

Use TUPLE when…

- Data should never change (config, constants)
- Returning multiple values from a function
- Using as a dictionary key
- Storing structured records (x, y, z)
- You need a slightly faster read-only list

Use DICT when…

- You need key → value lookups
- Counting occurrences (word frequency)
- Grouping data by category
- Caching computed results (memoization)
- Representing JSON/config data

Use SET when…

- You need fast membership testing (in operator)
- Removing duplicates from a collection
- Computing intersections / unions of groups
- Tracking visited nodes (BFS/DFS)
- Any scenario requiring unique elements only

Time Complexity & Performance Summary

Operation	list	tuple	dict	set
Access by index/key	O(1)	O(1)	O(1) avg	N/A
Search (x in ...)	O(n)	O(n)	O(1) avg	O(1) avg
Insert / Add	O(1) append	N/A	O(1) avg	O(1) avg
Delete	O(n)	N/A	O(1) avg	O(1) avg
Iterate all	O(n)	O(n)	O(n)	O(n)
Length (len)	O(1)	O(1)	O(1)	O(1)
Sort	O(n log n)	N/A	N/A	N/A
Copy	O(n)	O(n)	O(n)	O(n)

O(1) = Constant

O(n) = Linear

O(n log n) = Linearithmic

N/A = Not supported

Quick Reference — Cheat Sheet

```
# ── LIST ──────────────────────────────
lst = [3,1,4,1,5]
lst.append(9)        lst.pop()
lst.insert(0,0)      lst.remove(1)
lst.sort()           lst.reverse()
sorted(lst)          len(lst)
lst[1:3]             lst[::-1]
lst.index(4)         lst.count(1)
[x**2 for x in lst if x > 2]

# ── TUPLE ─────────────────────────────
tup = (1, 2, 3)
a, b, c = tup        # unpack
first, *rest = tup   # extended unpack
tup + (4,)           # concatenate
tup * 2              # repeat
```

```
# ── DICT ──────────────────────────────
d = {"a":1, "b":2}
d["c"] = 3           del d["a"]
d.get("x", 0)        d.pop("b")
d.keys()             d.values()
d.items()            d.update({"d":4})
{**d, "e":5}         d | {"e":5}
d.setdefault("f",0)
{k:v*2 for k,v in d.items()}

# ── SET ───────────────────────────────
s = {1, 2, 3}
s.add(4)             s.discard(2)
s | {5,6}            s & {2,3,9}
s - {1}              s ^ {1,9}
s <= {1,2,3,4}       s >= {1}
{x%3 for x in range(10)}
```

📌 Golden Rule: If you're checking membership frequently → set/dict. If order matters → list/tuple. If data is fixed → tuple. If you need key-value → dict.

Summary

list

Your go-to for ordered, changeable collections. Master append, slicing, and list comprehensions.

tuple

Lightweight & immutable. Ideal for fixed data, function returns, and as dict keys.

dict

The workhorse of Python. O(1) key lookup makes it perfect for mappings, counters, and configs.

set

Power tool for uniqueness and membership. Union, intersection, and difference in one line.

Master these four structures and you'll write cleaner, faster Python.

Python — Advanced Features

Comprehensions & Generators

List/dict/set comprehensions, generator expressions for memory-efficient iteration

Decorators & Context Managers

Higher-order functions, @property, @staticmethod, with statement patterns

Type Hints & Dataclasses

PEP 484 type annotations, @dataclass, Protocol for structural subtyping

Python

```python
# Generator + type hints + dataclass
from dataclasses import dataclass
from typing import Iterator

@dataclass
class Node:
    val: int
    children: list["Node"]

def traverse(root: Node) -> Iterator[int]:
    yield root.val
    for child in root.children:
        yield from traverse(child)

# Lazy evaluation — memory efficient
tree = Node(1, [Node(2, []), Node(3, [])])
result = list(traverse(tree))  # [1, 2, 3]
```

Key Concepts

- GIL & asyncio concurrency
- Magic methods (__dunder__)
- Metaclasses & ABCs
- functools: lru_cache, partial
- itertools: chain, islice, product
- Walrus operator :=

Part-2

Python Basics

LeetCode Problem
"Plus One"

Course Activities
Github repository
Discussion Forum

LeetCode Practice

Master Programming Skills with Python · C++ · Rust

2500+	50+	10M+
Problems	Companies	Users

Difficulty Tiers

- Easy
- Medium
- Hard

Arrays · Strings · Trees · Graphs · DP · Greedy · Divide & Conquer · Two Pointers · Sliding Window

Two Sum

PUAP01_Python_quick_tutorial.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

指令 程式碼 文字 全部執行 複製到雲端硬碟 連線 分享

```
instr = input('Please input your score: ')
score = int(instr) #Integer
print(score) # print the value of the variable
print(type(score)) # print the type of the variable
```

```
Please input your score: 88
88
<class 'int'>
```

Arithmetic Operators and Expressions Arithmetic Operators

```
+ - * / #Add, subtract, multiply and divide
** #power
// #business
% #remainder
```

```
print(1 + 2)
print(3 - 4)
```

變數 終端機

https://leetcode.com/problems/two-sum/description/
https://leetcode.cn/problems/two-sum/description/

Plus One

Problem List | Submit | Registe

Description | Editorial | Solutions | Submissions

66. Plus One

Easy | Topics | Companies

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the `i^th` digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading `0`'s.

Increment the large integer by one and return *the resulting array of digits*.

Example 1:

```
Input: digits = [1,2,3]
Output: [1,2,4]
Explanation: The array represents the integer 123.
Incrementing by one gives 123 + 1 = 124.
Thus, the result should be [1,2,4].
```

Example 2:

```
Input: digits = [4,3,2,1]
Output: [4,3,2,2]
```

Code

Python3 • Auto

```python
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:

```

Saved

You need to log in / sign up to run or submit

Testcase | Test Result

You must run your code first

https://leetcode.com/problems/plus-one/description/
https://leetcode.cn/problems/plus-one/description/

LeetCode

Problem Solving

#1 Two Sum

#66 Plus One

Hash Maps · Array Traversal · Carry Propagation

Brute Force → Optimal · Step-by-step walkthroughs · Complexity Analysis

Two Sum — Problem Statement

#1 Easy

Problem:

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target. You may assume each input has exactly one solution, and you may not use the same element twice.

Example 1

Input:

`nums = [2, 7, 11, 15], target = 9`

Output:

[0, 1]

nums[0] + nums[1] = 2 + 7 = 9

Example 2

Input:

`nums = [3, 2, 4], target = 6`

Output:

[1, 2]

nums[1] + nums[2] = 2 + 4 = 6

Example 3

Input:

`nums = [3, 3], target = 6`

Output:

[0, 1]

nums[0] + nums[1] = 3 + 3 = 6

Constraints: $2 \le$ nums.length $\le 10^4$ · $-10^9 \le$ nums[i] $\le 10^9$ · $-10^9 \le$ target $\le 10^9$ · Only one valid answer exists

Two Sum — Problem Analysis (UMPIRE Framework)

U Understand

Find 2 indices i, j where nums[i] + nums[j] = target. Return [i, j]. Indices, not values.

M Match

Array problem needing pair lookup. Matches 'Two Pointer' or 'Hash Map' pattern.

P Plan

Option A: Brute force $O(n^2)$ — try every pair. Option B: Hash map $O(n)$ — store complement.

I Implement

For each num, check if target−num is already in our hash map. If yes → found. If no → store num.

R Review

Edge cases: duplicate values ([3,3] target=6), negative numbers, target=0.

E Evaluate

Hash map: $O(n)$ time, $O(n)$ space. Brute force: $O(n^2)$ time, $O(1)$ space.

Two Sum — Brute Force Approach

$O(n^2)$ — Slow

Algorithm

1. **Step 1:** Use two nested for loops
2. **Step 2:** Outer loop: index i = 0 to n-2
3. **Step 3:** Inner loop: index j = i+1 to n-1
4. **Step 4:** Check if nums[i] + nums[j] == target
5. **Step 5:** If yes → return [i, j]

Drawback: $O(n^2)$ time — tries ALL pairs. For n=10,000 that's 50 million checks!

```python
def two_sum_brute(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]

# Time:  O(n²)  Space: O(1)
# For n=10000: ~50,000,000 iterations
```

Two Sum — Brute Force Walkthrough [2, 7, 11, 15], target=9

nums =

2	7	11	15
0	1	2	3

2 + 7 = 9 ✓

i	j	nums[i]	nums[j]	Sum	== target?	Action
0	1	2	7	9	✓ YES!	`return [0, 1]`
0	2	2	11	13	✗ No	`continue...`
0	3	2	15	17	✗ No	`continue...`
1	2	7	11	18	✗ No	`continue...`

✓ Found on iteration 1! Return [0, 1] — But in worst case, we'd check n(n-1)/2 = all pairs.

Two Sum — Hash Map: The Key Insight

O(n) — Optimal

Key Insight:

Instead of looking forward for a pair, for each number x, ask: "Have I already seen target – x?"
This transforms a 2-pass $O(n^2)$ scan into a single O(n) pass using a hash map as memory.

Algorithm Logic

1. `Create empty hash map: seen = {}` # key=value, val=index
2. `For each element nums[i]:`
3. `  Compute complement = target – nums[i]`
4. `  If complement is in seen:` # O(1) lookup!
5. `    → Return [seen[complement], i]` # Found the pair!
6. `  Else: store seen[nums[i]] = i` # Remember it

```python
def two_sum(nums, target):
    seen = {}  # val → index

    for i, num in enumerate(nums):
        complement = target - num

        if complement in seen:
            return [seen[complement], i]

        seen[num] = i

# Time: O(n)  Space: O(n)
```

Two Sum — Hash Map Walkthrough [2, 7, 11, 15], target=9

nums =

2	7	11	15
0	1	2	3

target = 9

i = 0 **num = 2** `complement = target - num = 9 - 2 = 7`

seen before: `{}`

complement 7 not in {} → store 2

seen after: `{ 2: 0 }`

i = 1 **num = 7** `complement = target - num = 9 - 7 = 2`

seen before: `{ 2: 0 }`

complement 2 IS in seen[2]=0 → return [0, 1]

seen after: `{ 2: 0, 7: 1 }`

Answer: [0, 1] — Only 2 iterations needed vs. brute force's 6 checks for n=4!

Two Sum — Complete Solution & Edge Cases

```python
from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) ->
List[int]:
        seen = {}  # {value: index}

        for i, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], i]
            seen[num] = i

# Problem guarantees a solution, so no need for
return []
```

Edge Cases

Duplicate values

`[3, 3], target=6 → [0, 1]`

Works! i=0 stores 3→0. i=1 sees complement=3 in seen.

Negative numbers

`[-1, -2, -3, -4], target=-6 → [1, 3]`

-2 + -4 = -6. Hash map handles negatives fine.

Target = 0

`[-3, 3, 5], target=0 → [0, 1]`

complement = 0 - (-3) = 3. Works normally.

Min length (n=2)

`[5, 6], target=11 → [0, 1]`

Only possible pair. Returns immediately at i=1.

Two Sum — Complexity Analysis

Brute Force

Time: $O(n^2)$

Space: $O(1)$

Pros:

✓ No extra memory needed

✓ Simple to implement

Cons:

✗ Very slow for large n

✗ 50M ops for n=10,000

When: Only use for tiny arrays (n < 100)

Hash Map (Optimal)

Time: $O(n)$

Space: $O(n)$

Pros:

✓ Single pass through array

✓ Scales to 10M+ elements

Cons:

✗ Uses O(n) extra memory

✗ Hash collisions (rare)

When: Always prefer this approach

Performance comparison at n=10,000: Brute Force = ~50,000,000 ops vs Hash Map = ~10,000 ops → 5000× faster!

Two Sum — Summary & Key Takeaways

The Core Pattern

For each element, check if its complement (target − x) was seen before. This is the Hash Map Complement Pattern — memorize it!

Hash Map = O(1) Lookup

Python dict uses a hash table internally. The 'in' operator on a dict/set is O(1) average vs O(n) for a list.

One-Pass is Enough

We don't need to build the hash map first then search. We can check & insert in the same loop.

This Pattern Appears In

Two Sum II, Three Sum, Four Sum, Subarray Sum Equals K, Intersection of Arrays, and many more.

```
seen = {}; for i, x in enumerate(nums): if target-x in seen: return [seen[target-x], i]; else: seen[x] = i
```

Plus One — Problem Statement

#66 Easy

Problem:

You are given a large integer represented as an integer array digits, where each digits[i] is the i-th digit of the integer. The digits are ordered from most significant to least significant. Add one to the integer and return the resulting array of digits.

Example 1

Input: [1, 2, 3]

Output: [1, 2, 4]

123 + 1 = 124

Example 2

Input: [4, 3, 2, 1]

Output: [4, 3, 2, 2]

4321 + 1 = 4322

Example 3

Input: [9]

Output: [1, 0]

9 + 1 = 10 ← carry!

Example 4

Input: [9, 9, 9]

Output: [1, 0, 0, 0]

999 + 1 = 1000 ← all carry!

Constraints: 1 ≤ digits.length ≤ 100 · 0 ≤ digits[i] ≤ 9 · No leading zeroes (except the number 0 itself)

Plus One — Understanding Carry Propagation

Key Insight:

Adding 1 to a digit either (a) ends immediately if digit < 9, or (b) creates a carry that propagates left. Start from the last digit and work backwards — just like manual addition.

Case 1: No Carry

`[1, 2, 3] → [1, 2, 4]`

Input:

1	2	3

Output:

1	2	4

Last digit (3) + 1 = 4 < 10. Just set it. Done!

Case 2: One Carry

`[1, 2, 9] → [1, 3, 0]`

Input:

1	2	9

Output:

1	3	0

9+1=10 → set to 0, carry 1 left. 2+1=3 < 10. Done!

Case 3: All Carry

`[9, 9, 9] → [1, 0, 0, 0]`

Input:

9	9	9

Output:

1	0	0	0

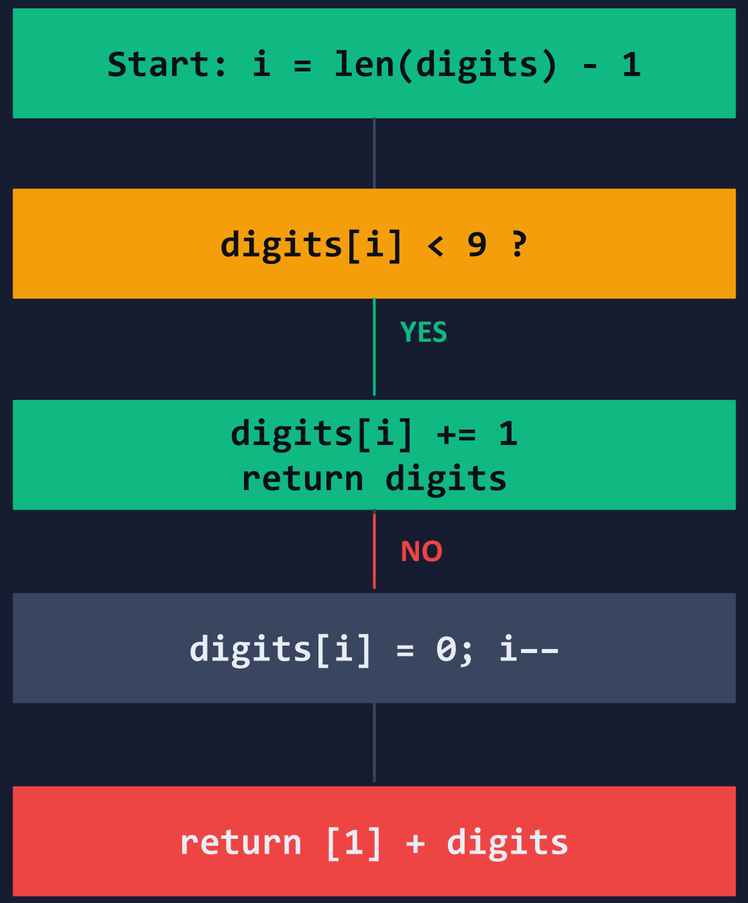
All 9s! Each becomes 0, carry propagates. Must prepend 1.

Plus One — Algorithm Design

Algorithm Logic

1. Traverse from last digit (right to left)
2. If current digit < 9:
3. → Add 1, return digits immediately
4. If current digit == 9:
5. → Set to 0, carry propagates left
6. If loop ends (all were 9s):
7. → Insert 1 at front: [1] + digits

Decision Flow



Plus One — Walkthrough: [1, 2, 9] (Carry Case)

digits =

1	2	9
		Step 1

Start: i=2 → digits[2] = 9 → Is 9 < 9? NO → Set to 0, carry

```
digits[2] = 9 → set to 0, carry
```

1	3	0
	Step 2	

i=1 → digits[1] = 2 → Is 2 < 9? YES → digits[1] += 1 → 3 → Return!

Result: [1, 3, 0] · 129 + 1 = 130 ✓ Stopped at i=1 because digits[1]+1 = 3 < 10

Plus One — Walkthrough: [9, 9, 9] (All-Carry Case)

Step	i	digits[i]	Check	Action	State
1	2	9	9 < 9? NO	Set digits[2] = 0	[9, 9, 0]
2	1	9	9 < 9? NO	Set digits[1] = 0	[9, 0, 0]
3	0	9	9 < 9? NO	Set digits[0] = 0	[0, 0, 0]
4	-1	-	Loop ends	**Prepend 1 → [1, 0, 0, 0]**	[1, 0, 0, 0]

Array Evolution:

start	step 1	step 2	step 3	final
9 9 9	9 9 0	9 0 0	0 0 0	1 0 0 0

Special case: all 9s! After loop, digits = [0,0,0] → prepend 1 → [1,0,0,0] representing 999+1=1000

Plus One — Complete Solution

```python
from typing import List

class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        # Traverse from last digit to first
        for i in range(len(digits) - 1, -1, -1):
            if digits[i] < 9:
                digits[i] += 1
                return digits

            # Carry: set current digit to 0
            digits[i] = 0

        # All digits were 9, need extra digit
        return [1] + digits
```

Why range(n-1, -1, -1)?

This iterates indices from the last position to 0 (inclusive), stepping by -1 — i.e., right to left. Equivalent to:

```python
# Alternative:
for i in range(n-1, -1, -1)
```

One-liner (less readable):

```python
def plusOne(self, d):
    n = int("".join(map(str,d))) + 1
    return [int(c) for c in str(n)]
```

Plus One — Complexity & Edge Cases

Complexity Analysis

Time Complexity: O(n)

Best case O(1) — last digit isn't 9. Worst case O(n) — all digits are 9.

Space Complexity: O(1)

Input	Output	Case	Explanation
`[0]`	`[1]`	Single zero	0 < 9, so just increment. No carry needed.
`[9]`	`[1, 0]`	Single 9	9 → 0 with carry, loop ends, prepend 1.
`[1, 0, 0]`	`[1, 0, 1]`	Trailing zeros	Last digit 0 < 9, just increment. Done at step 1.
`[9, 9]`	`[1, 0, 0]`	Two nines	Both become 0, loop ends, prepend 1.
`[1, 9, 9]`	`[2, 0, 0]`	Mixed carry	9→0, 9→0, then 1+1=2 < 10, stop.
`[8, 9, 9, 9]`	`[9, 0, 0, 0]`	Long carry	Three 9s become 0, then 8+1=9 < 10, stop.

Plus One — Summary & Key Takeaways

↩ Traverse Right to Left

Addition with carry always starts from the least significant digit. Reverse traversal is the natural model for carry propagation problems.

0 The All-9s Edge Case

Critical! If all digits are 9 (e.g., 999), the answer needs one more digit (1000). The [1] + digits trick handles this elegantly.

Early Return = Efficiency

The moment we add 1 without overflow, we return immediately. Best case is O(1) when the last digit is not 9.

Related Patterns

Add Binary (Leetcode #67), Multiply Strings (#43), Add Two Numbers (#2 linked list). All use the same carry propagation logic.

```
for i in range(len(d)-1, -1, -1): if d[i]<9: d[i]+=1; return d  else: d[i]=0  →  return [1]+d
```

Comparison: Two Sum vs Plus One

Property	Two Sum #1	Plus One #66
Category	Array + Hash Map	Array Math
Core Pattern	`Complement lookup (target - x in seen)`	`Carry propagation (right to left)`
Data Structure	Hash Map (dict)	Array (in place)
Time Complexity	O(n)	O(n) worst, O(1) best
Space Complexity	O(n) for hash map	O(1) in place
Key insight	`"Have I seen the complement before?"`	`"Is current digit < 9? If not, carry."`
Tricky case	`Duplicate values [3,3]`	`All 9s → [1,0,...,0]`
LeetCode difficulty	Easy	Easy

Key Takeaways

Two Sum & Plus One

#1 Two Sum — Hash Map Pattern

- Brute force $O(n^2)$ tries all pairs — never do this in interviews
- Hash map O(n): for each x, check if target−x was seen before
- One pass: check the map and insert in the same loop
- Memorize: complement = target − current
- Applies to: 3Sum, 4Sum, Subarray Sum K, and more

#66 Plus One — Carry Propagation

- Traverse from right (last digit) to left (first digit)
- If digit < 9: increment, return immediately — O(1) best case
- If digit == 9: set to 0 and continue the loop (carry)
- After loop ends: all were 9s → prepend 1 with [1] + digits
- Applies to: Add Binary, Add Two Numbers, Multiply Strings

Practice these patterns and you will recognize them in 50+ other LeetCode problems.

Part-3

Python Basics

LeetCode Problem
"Two Sum"

Course Activities
Github repository
Discussion Forum

Creating a

GitHub Repository

Remote Storage

Host code in the cloud

Collaboration

Work with any team

Version Control

Track every change

Step-by-Step Guide for Beginners · git init → push → collaborate

1 Sign Up & Set Up Your Account

Go to github.com and register

Enter your details

Username, email address, and a strong password. Your username will be part of your public URL.

Verify your email

GitHub sends a confirmation email. Click the link to activate your account before proceeding.

Choose a plan

Free tier is sufficient for most students. Public repos are unlimited; private repos allowed on free plan.

Enable 2FA (recommended)

Two-factor authentication adds security. Use an authenticator app like Google Authenticator.

Your GitHub profile URL will be:

`github.com/your-username`

Username Tips

- Use lowercase letters and hyphens only
- Keep it professional — it appears on your resume
- Avoid numbers at the end (e.g., user123)
- Make it short and memorable

2 Create a New Repository

github.com/new

Create a new repository

Owner

your-username

Repository name *

my-first-repo

Description (optional)

My first GitHub project

Visibility

Public | Private

Repository Name

Must be unique within your account. Use lowercase, hyphens, no spaces. e.g. my-cool-project

Public vs Private

Public: anyone can view. Private: only you and invited collaborators. Students often use Public for portfolios.

Description

Optional but recommended. A brief description helps others understand your project's purpose.

Tip

Click the green '+' button (top-right on GitHub) or go to github.com/new to start creating a repo.

3 Initialize Repository Files

README.md

Highly Recommended

The front page of your repository. Written in Markdown, it describes what the project does, how to install it, how to use it, and how to contribute.

```
# My Project

A brief description.

## Installation
```bash
npm install
```
```

.gitignore

Recommended

Tells Git which files to NOT track (e.g., node_modules/, .env, __pycache__). GitHub offers templates for Python, Node.js, Java, etc.

```
# Python example
__pycache__/
*.pyc
.env
venv/
.DS_Store
```

LICENSE

Optional

Defines how others may use your code. MIT License is the most permissive and widely used. Required if you want others to legally reuse your work.

```
MIT License

Copyright (c) 2025
Your Name

Permission is hereby
granted, free of charge...
```

4 Clone the Repository to Your Computer

Copy the Clone URL from GitHub: `https://github.com/username/my-first-repo.git`

HTTPS vs SSH

HTTPS (Recommended for beginners)

`https://github.com/user/repo.git`

Use your GitHub username & Personal Access Token as password.

SSH (For advanced users)

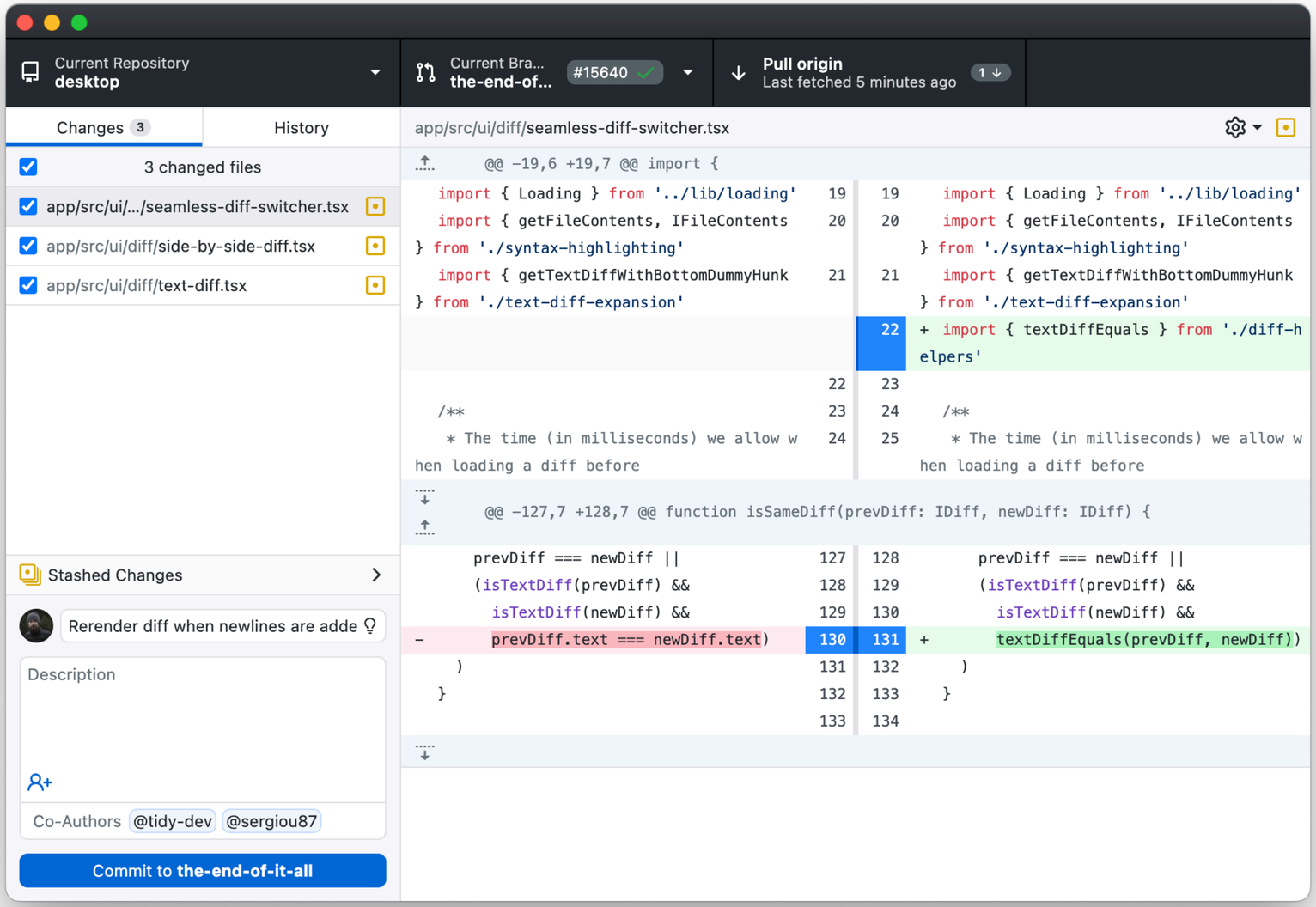
`git@github.com:user/repo.git`

Requires SSH key setup but no password prompt after configuration.

Terminal

```
$ git clone https://github.com/
    username/my-first-repo.git
  Cloning into 'my-first-repo'...
  remote: Enumerating objects: 3...
  Receiving objects: 100% (3/3)
$ cd my-first-repo
$ ls
  README.md  .gitignore  LICENSE
```

4 GitHub Desktop



https://desktop.github.com/download/

Explore **Topics** Trending Collections Events GitHub Sponsors

leetcode-problems-solutions

Star

Here are 39 public repositories matching this topic...

Language: All Sort: Most stars

Bishalsarang / Leetcode-Questions-Scraper

Star 160

Code Issues Pull requests

Scrape Algorithm Questions from leetcode and generate html and epub file

scraper leetcode scraping scrapper scraping-websites leetcode-questions leetcode-downloader leetcode-problems-solutions leetcode-scrapper download-leetcode leetcode-offline leetcode-pdf leetcode-epub leetcode-download leetcode-save

Updated on Dec 24, 2023 Python

https://github.com/topics/leetcode-problems-solutions